

My SQL Queries

SQL (Structural Query Language):

Programming language designed for managing data in relational database.

SQL operations:

SELECT Data, INSERT Data, UPDATE Data
JOIN Different Tables

SELECT Basics

SELECT Basics

On workbench:

⇒ Write a query to get all of the users:

```
SELECT *  
FROM users;
```

⇒ Write a query to get the first name / last name from users:

```
SELECT first-name, last-name  
FROM users
```

SELECT with conditions

⇒ Write a query to get the first name of users with an ID of 2:

```
SELECT first-name  
FROM users  
WHERE id=2
```

⇒ Write a query to get the users with IDs less than or equal to 3?

```
SELECT *  
FROM users  
WHERE id <= 3;
```

⇒ Write a query to get all users with first-name ending with "e"?

```
SELECT *  
FROM users  
WHERE first-name LIKE "%e"
```

⇒ Write a query to get all the users with first-name starting with "K"?

```
SELECT *  
FROM users  
WHERE first-name LIKE "K%"
```

Summary:

→ SELECT *
FROM users

gets the whole table

→ SELECT first-name
FROM users
gets all first names in users

→ SELECT first-name
FROM users
WHERE id=5

Select with condition

→ get first name that

- ends with an e → "%e"
- Starts with an e → "e%"

SELECT with Sorting

→ get all the users with the youngest users at the top of the table?

SELECT *

FROM users

ORDER BY birthday DESC / ASC

→ get all users first names in alphabetical order:

SELECT first-name

FROM users

ORDER BY first-name

Default of ORDER BY is ASC

INSERT Basics

The command to insert records in table:

INSERT INTO table-name (column-name1,
column-name2)

VALUES ("column1-value", "column2-value")

UPDATE Basics

The command for getting / updating records is as follows:

✗

- UPDATE table-name SET column-name1 = "some val"

- column-name2 = "another-val" WHERE condition

NOTE

If the WHERE condition is not added to the UPDATE statement, the changes will be applied to every record in the table.

DELETE Records

DELETE FROM table-name WHERE condition

Quiz Questions

1) You can use the BETWEEN operator for:

A. finding all the rows between 2 values with type DATETIME

B. finding all the rows between 2 values with the type VARCHAR

C. finding all the rows between 2 values with type INT.

2) "ORDER BY in DESC" will show the most recent record first (this assumes that we have created a table using the proper conventions with id being a primary key: True

3) Where do we apply my SQL functions?
any place within a query where you are making reference to a column.

4) 'SELECT * FROM customers JOIN orders

ON customer.id = orders.customer-id;

→ Only the records orders where customer-id has a match for the id in the customers table

JOINS

1) One to One

SELECT * FROM customers

JOIN address ON address.id = customers.address-id

customers

id

address-id

addresses

id

street

2) One to Many

SELECT * FROM orders

JOIN customers ON customers.id = orders.customer-id

orders

id

customer-id

customers

id

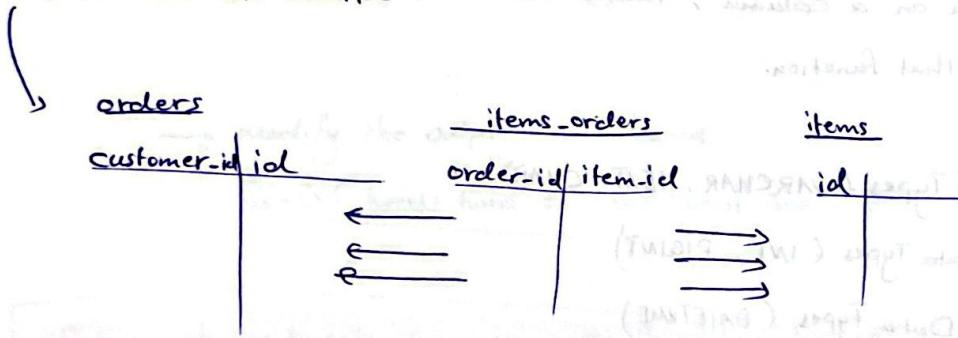
address-id

3) Many to Many

SELECT * FROM orders

JOIN items_orders ON orders.id = items_orders.order-id

JOIN items ON items.id = items_orders.item-id



Grouping Results

• GROUP BY is used in SQL to combine rows that have the same values in one or more columns into a single row

• Once grouped, you can apply aggregate (grouping) functions to summarize the data for each group.

Quiz

5) result of: 'SELECT DATE_FORMAT('2012-03-09 12:30', '%W %M %D %Y at %r')
as great-Time Friday March 9th 2012 at 12:30:00PM

Common Aggregate Functions

- AVG (column) → returns the average value of a column.
- COUNT (column) → counts the number of values in a column.
- GROUP CONCAT (column) → the concatenation of a column's value.
- MAX (column) → the largest value of a column.
- MIN (column) → the minimum value of a column.
- SUM (column) → the sum of all the values in a column.

Functions

• When calling a function on a column, make sure that column is the appropriate Data Type for that function.

→ Text Functions Data Types (VARCHAR, TEXT, CHAR)

→ Numeric Functions Data Types (INT, BIGINT)

→ Date & Time Functions Data Types (DATETIME)

Usage Syntax:

```
SELECT Function (column-name)  
FROM table-name;
```

Examples:

```
SELECT ROUND (AVG (age), 1) FROM students
```

Check the rest of the functions on miro <Functions Slide>

LEFT JOIN

From the example in the material:

get all tweets from the user ID of 1:

```
SELECT *  
FROM users  
LEFT JOIN tweet  
ON users.id = tweets.user-id  
WHERE users.id = 1;
```

Copy

users
users.id

tweets
tweets.user-id

id		id	tweet	user_id
1		1		1
1		2		1
1		3		1

→ grab the tweets by:

```
SELECT tweets.tweet
```

tweet

→ modify the output column name:

```
SELECT tweets.tweet as Kobe_tweets
```

Kobe_tweets

Return all the tweets the user with ID 2 has tagged as favorite

```
SELECT first_name, tweet
```

```
FROM users
```

```
LEFT JOIN faves
```

```
ON users.id = faves.user-id
```

```
LEFT JOIN tweets
```

```
ON faves.tweet-id = tweets.id
```

```
WHERE users.id = 1
```

users

faves

tweets

• get all the users following the user with ID 1:

```
SELECT users.first_name as followee, users2.first_name as follower
FROM users
LEFT JOIN follows
ON users.id = follows.followee_id
LEFT JOIN users as users2
ON users2.id = follows.follower_id
WHERE users.id = 1
```

• get all users who are not following the user with ID 2:

```
SELECT *
FROM users
WHERE users.id NOT IN (
  SELECT follower_id
  FROM follows
  WHERE followee_id = 2
) AND users.id != 2
```

View DB created

- 1) Select 
- 2) select local host
- 3) Select the refresh option next to schema if the created schema does not appear in the list.
- 4) Connect to the schema:
1) - double click on schema name
or - add: use <schema-name> on the top

Forward Engineering

- 1) Be sure the MySQL server is running on your OS.
- 2) Select Database → Forward Engineer.
- 3) Select generate-drop-schema → if you are updating a schema that you already Forward engineered otherwise leave unchecked.
- 4) Continue, save to file
- 5) move saved file to the project folder
- 6)

Forward Engineering progress
 → all of them has to be green

Schema → Definition of the database / tables
data types

Create database database-name;
Create schema database-name;

drop database database-name ⇒ delete database

CRUD → delete
create read update

بہی آگے query کیسے لکھیں؟
workbench میں ایسی ہی لکھیں
جس سے ڈیٹا بیس میں
کچھ تبدیلیاں آسکیں

How to create DB?
create database name;

use database-name;

How to delete DB?
drop database name;

and any query that I will write from
now on will be used in this DB

How to use DB?
use database-name;

how to write query in a specific DB?
use database-name

How to see DB?
show tables;

⇒ create table users (

id INT PRIMARY KEY NOT NULL AUTO-INCREMANT,

firstname NVCHAR(50) NOT NULL

idnum NVARCHAR(50) NOT NULL UNIQUE

)

show tables

pt062029

Tables

users

pressing this arrow will show table

id command

⇒ create table addresses (

id INT PRIMARY KEY NOT NULL Auto-INCREMANT

user-id INT NOT NULL

foreign key

connected to user-id

id INT

desc table-name;
description of the table

Notes from discussion

modify table:

→ alter table addresses drop / add / modify Column_name Varchar(90) not null
کمیته جدول

Add Foreign Key:

alter table addresses add constraint FK_user_id foreign key (user_id)
references users (id);
جواب FK

display table data? select first_name, last_name
from users;

Select all columns from the table? select *;
from users;

how to delete a specific user? / specific row?
delete from users where id = 9;

Delete all the table?

delete from users;

is command for deleting
all data from a table
table

INSERT data in the table?

insert into users (id, first_name, last_name)
values (32025, "Nour", "Ibrahim")

id is
auto-increment
لا يمكن ان يكون
القيمة

update user?

update users set age = 23

where id = ---